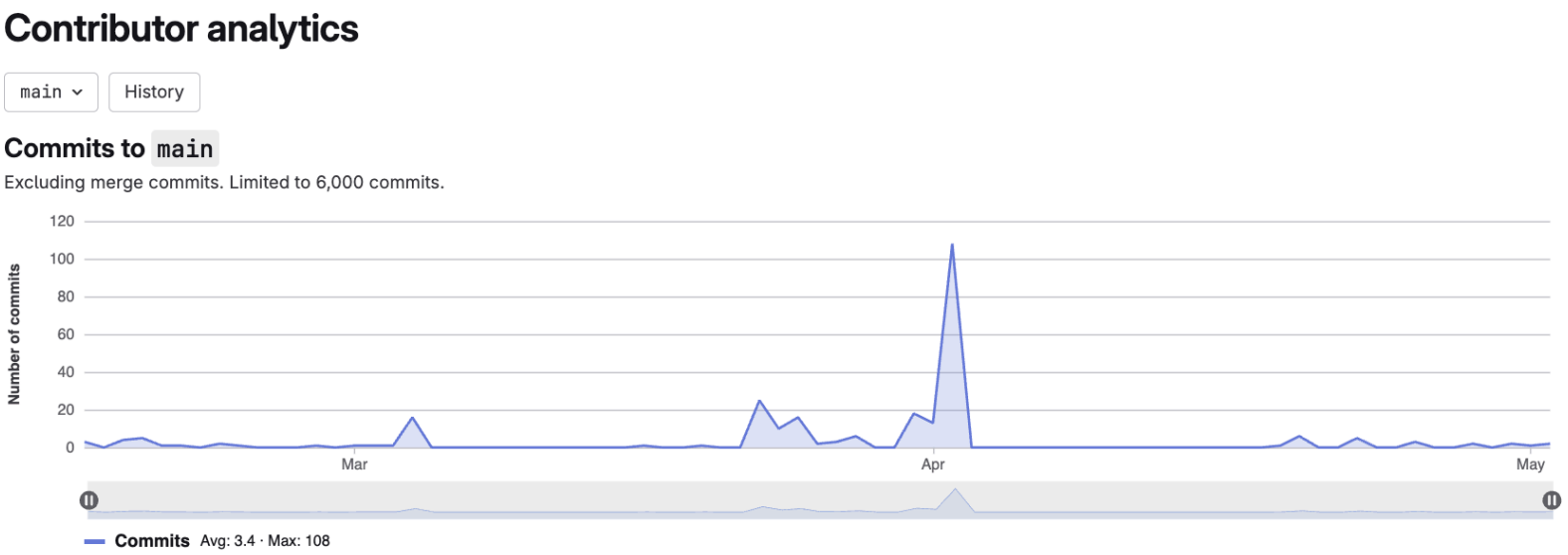
MONASH University

FIT3077 Software Engineering: Architecture and Design

(Sprint Three)

Wong Xin Thung (33592357)

1.0 Contributor Analytics



2.0 Brief of Extensions

In this sprint, the three extensions that have been chosen are to introduce a new god card – Zeus. The Zeus has a special ability which can build a block under itself, but building a 3rd-level block underneath does not qualify as a win. The second extension is Timer. Each player gets a fixed amount of time for “thinking” and executing their turns. Once a player has completed their turn, their clock pauses and the clock starts for the next player. If the clock of a player reaches zero, the player loses the game. In this sprint, the player timer was set with 5 minutes per round. Lastly, a SkidCard was added in this sprint as the last extension. Skipcard allows a player to skip the next player’s turn. SkipCard is a function card that every player has. Every player has one Skipcard. If they used their skip card, then they cannot use it a second time; they can only use it once.

3.0 UML Class Diagram

This section shows the iteration process of updating and refining the UML Class Diagram throughout the sprint, reflecting changes in design decisions, class responsibilities, and system relationships as development progresses. An extensive explanations and justifications of the classes showed in the UML Class Diagram are included under the section of Design Rationale. A clearer and a full picture of the UML Class Diagrams can be found via the draw.io link attached under the appendix section or in the repository of git.

3.1 UML Class Diagram - Iteration 1

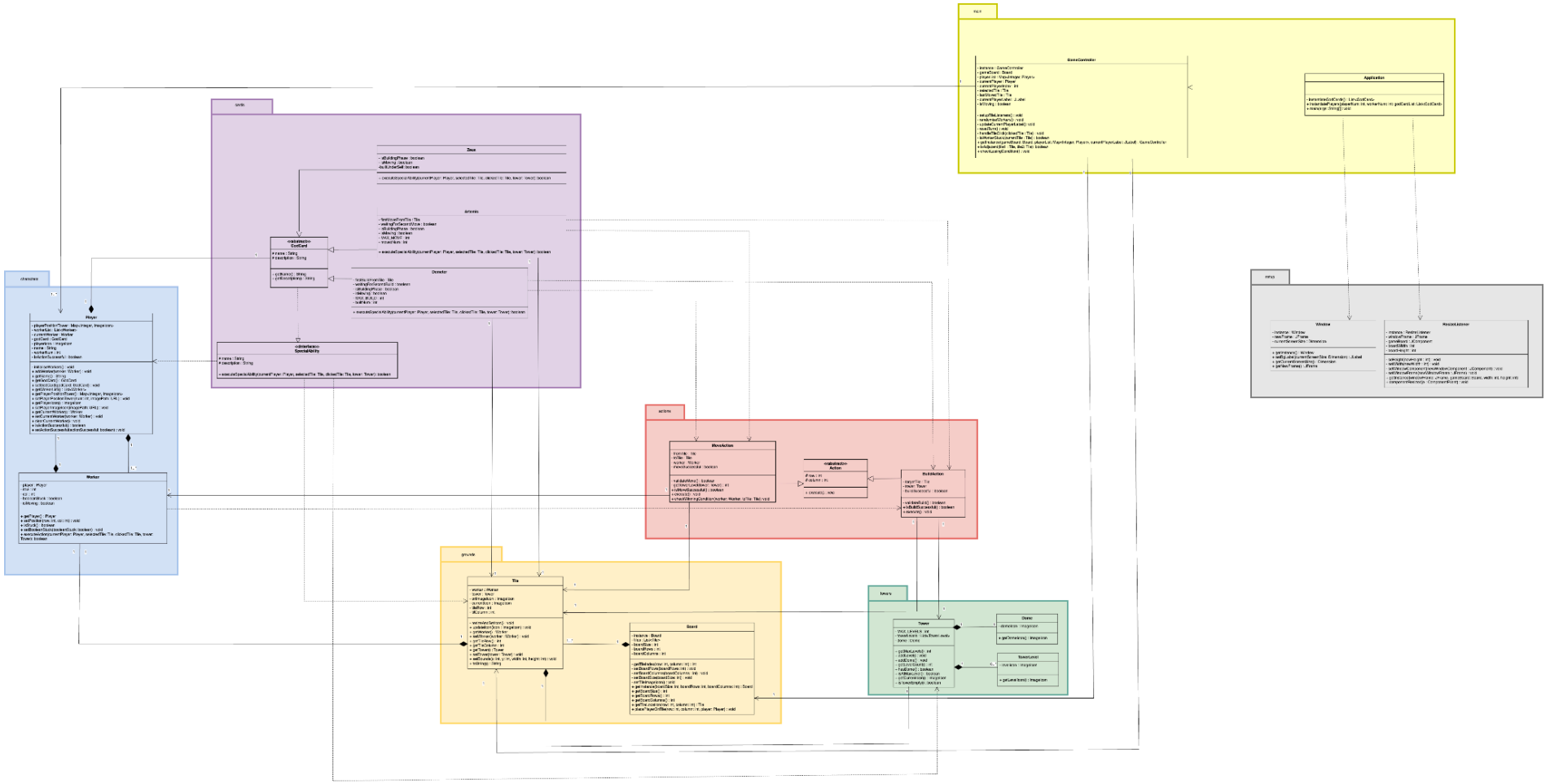


Diagram 2.1: This diagram shows the UML diagram for Iteration 1

3.2 UML Class Diagram - Iteration 2

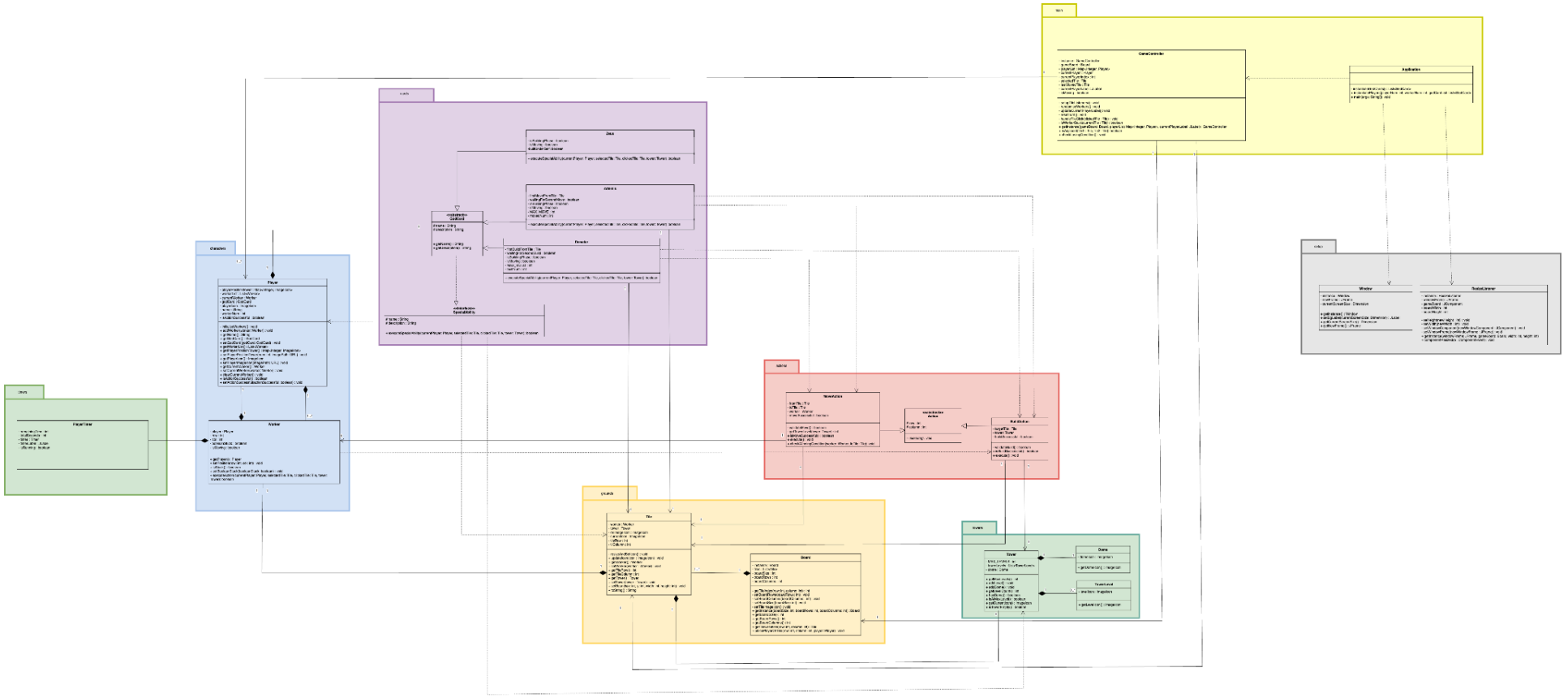


Diagram 2.2: This diagram shows the UML diagram for Iteration 2

3.3 UML Class Diagram - Iteration 3

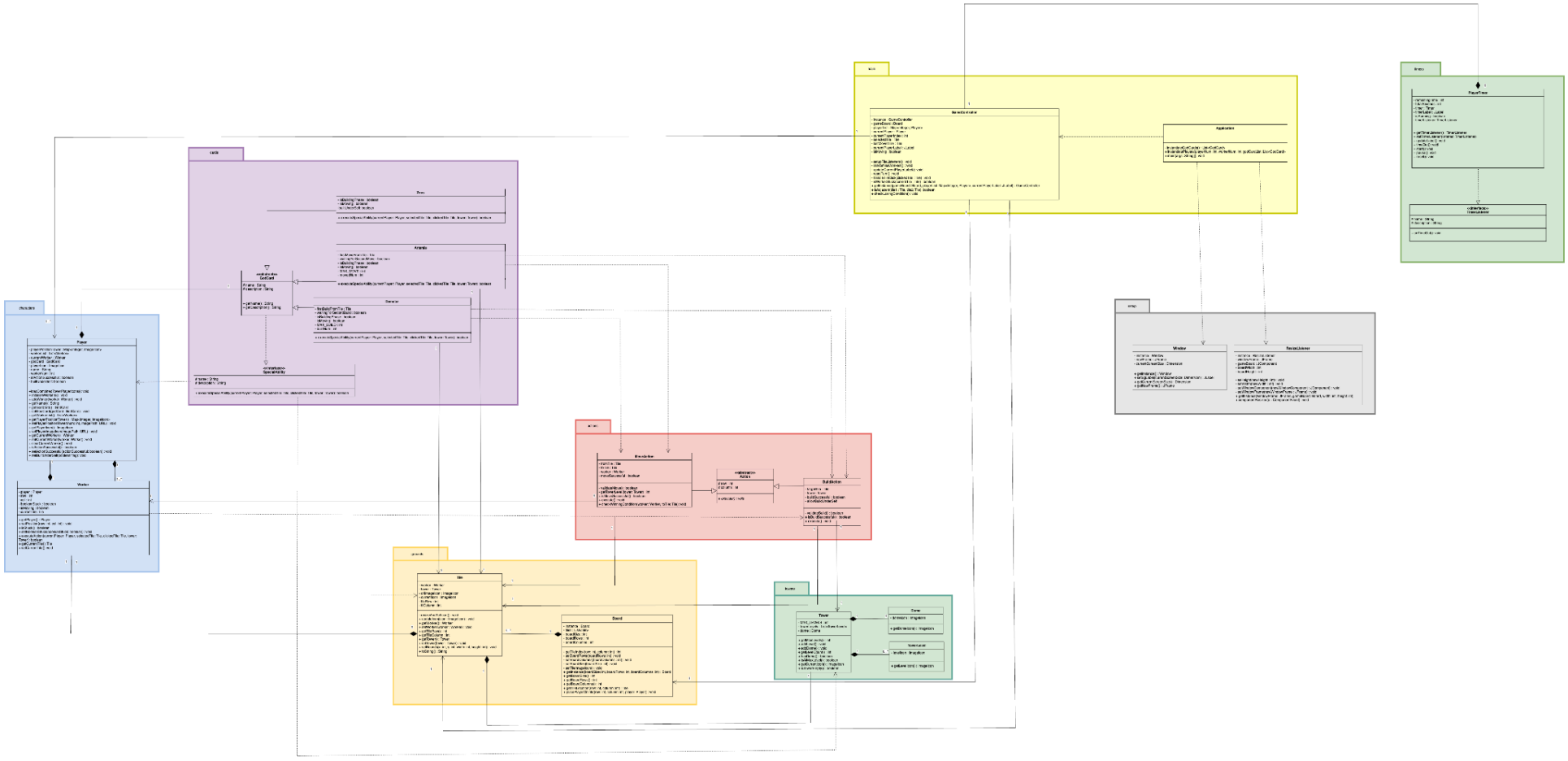


Diagram 2.3: This diagram shows the UML diagram for Iteration 3

3.4 UML Class Diagram - Iteration 4

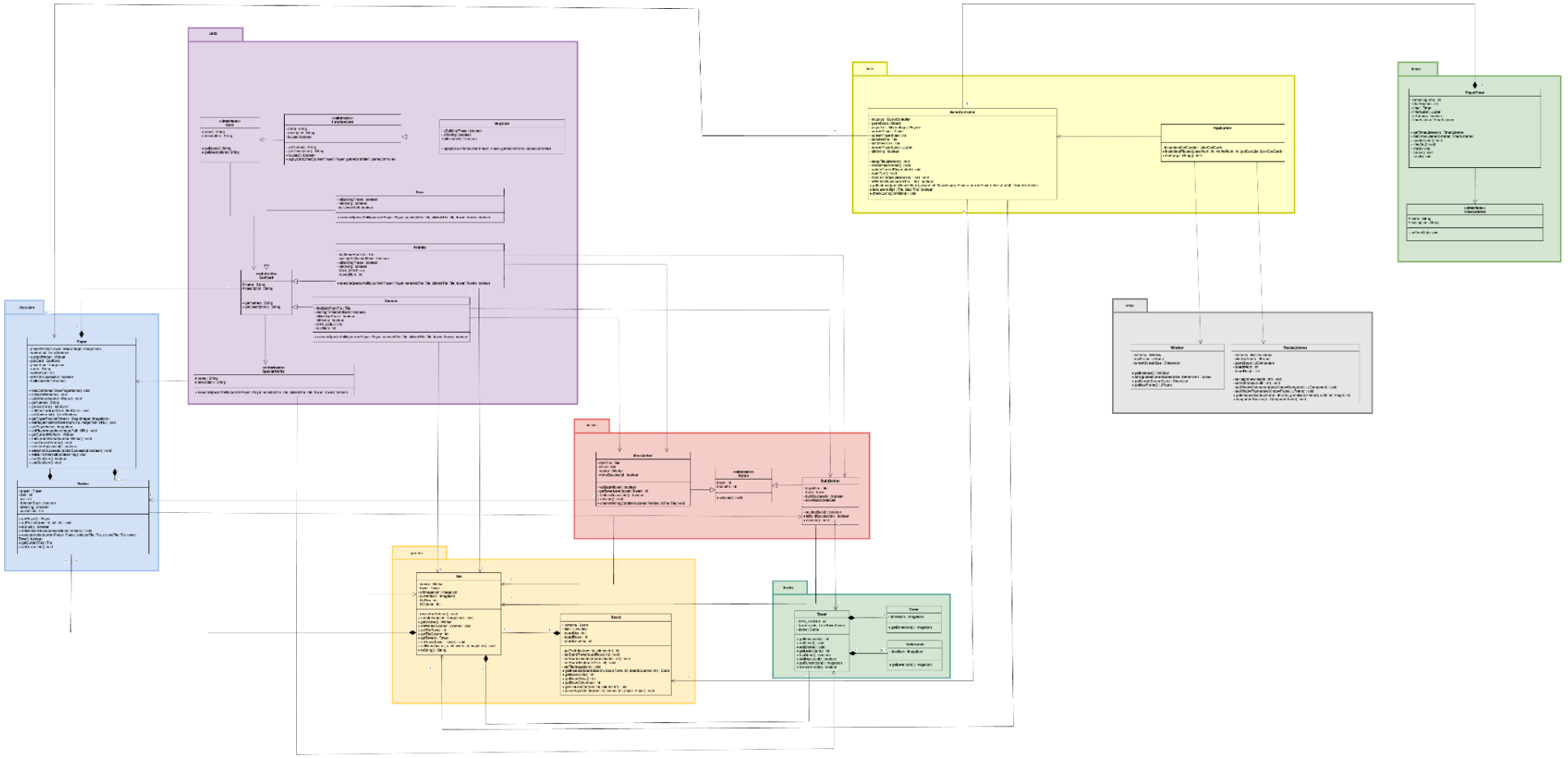


Diagram 2.4: This diagram shows the UML diagram from Iteration 4

3.5 UML Class Diagram - Iteration 5 (Finalised UML Class Diagram)

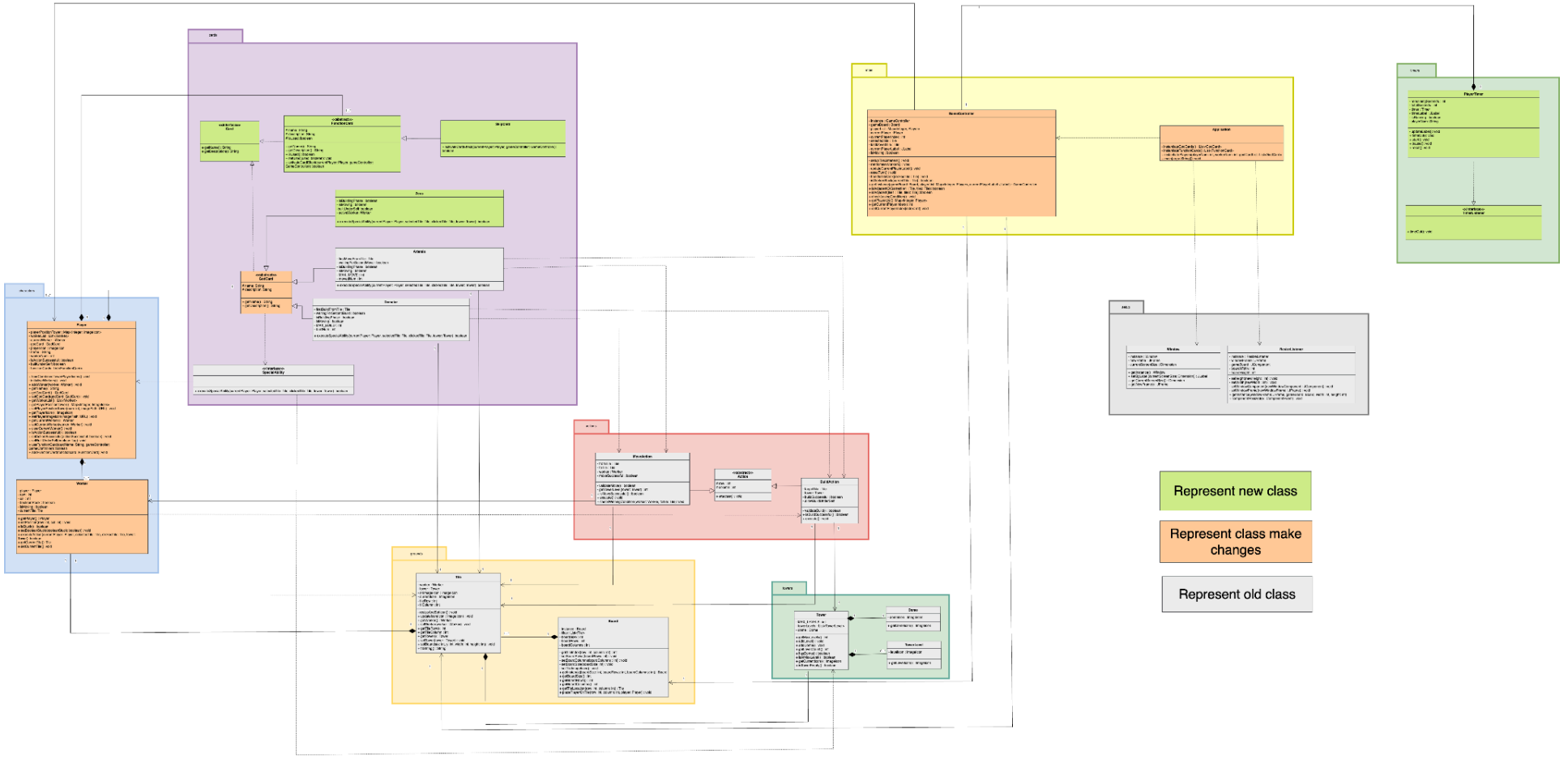


Diagram 2.4: This diagram shows the finalised UML diagram from Iteration 5 and comparison between Sprint 2 uml

3.6 Comparison of UML Diagrams Between Sprint 2 and Sprint 3

This section compares the design evolution between Sprint 2 and Sprint 3 using UML diagrams.

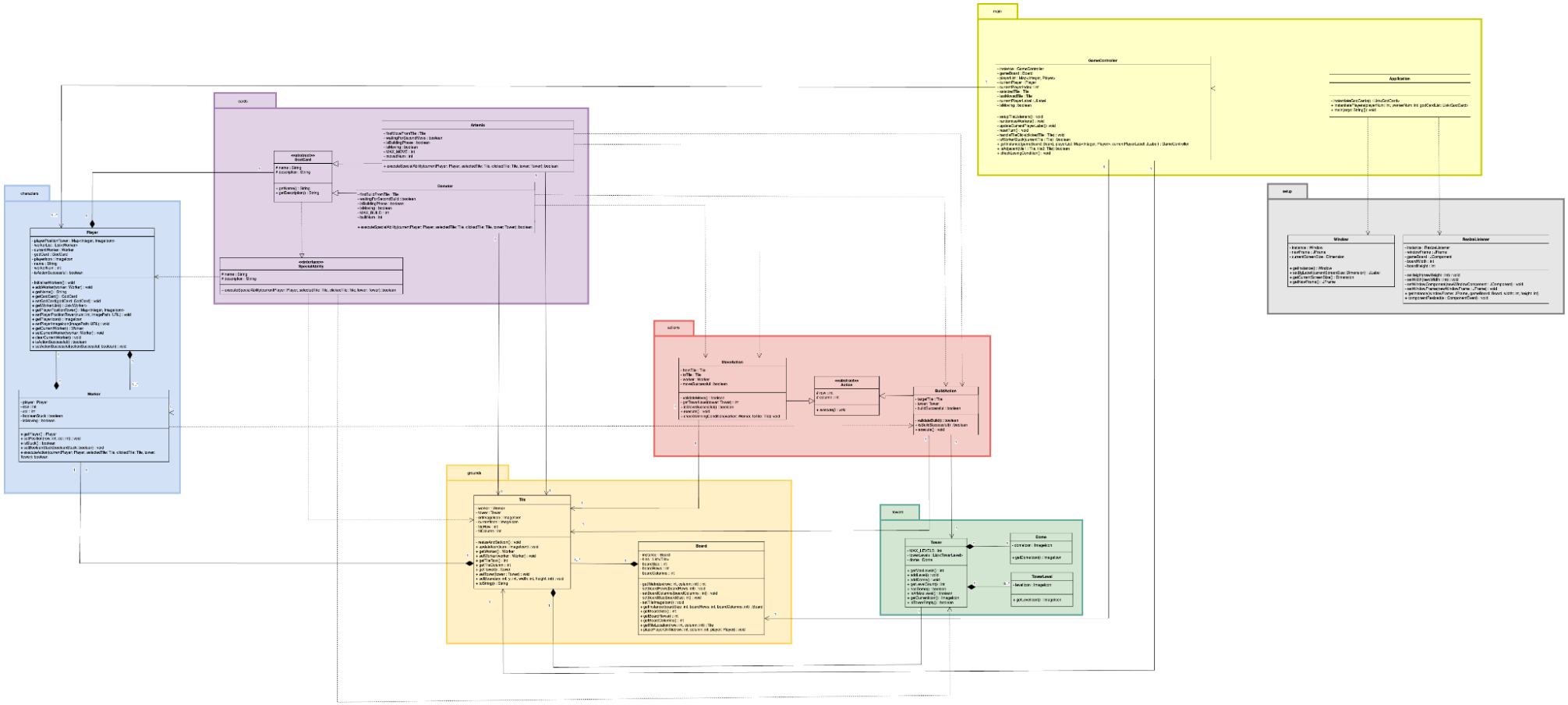


Diagram 3.6.1: UML Class Diagram - Sprint 2

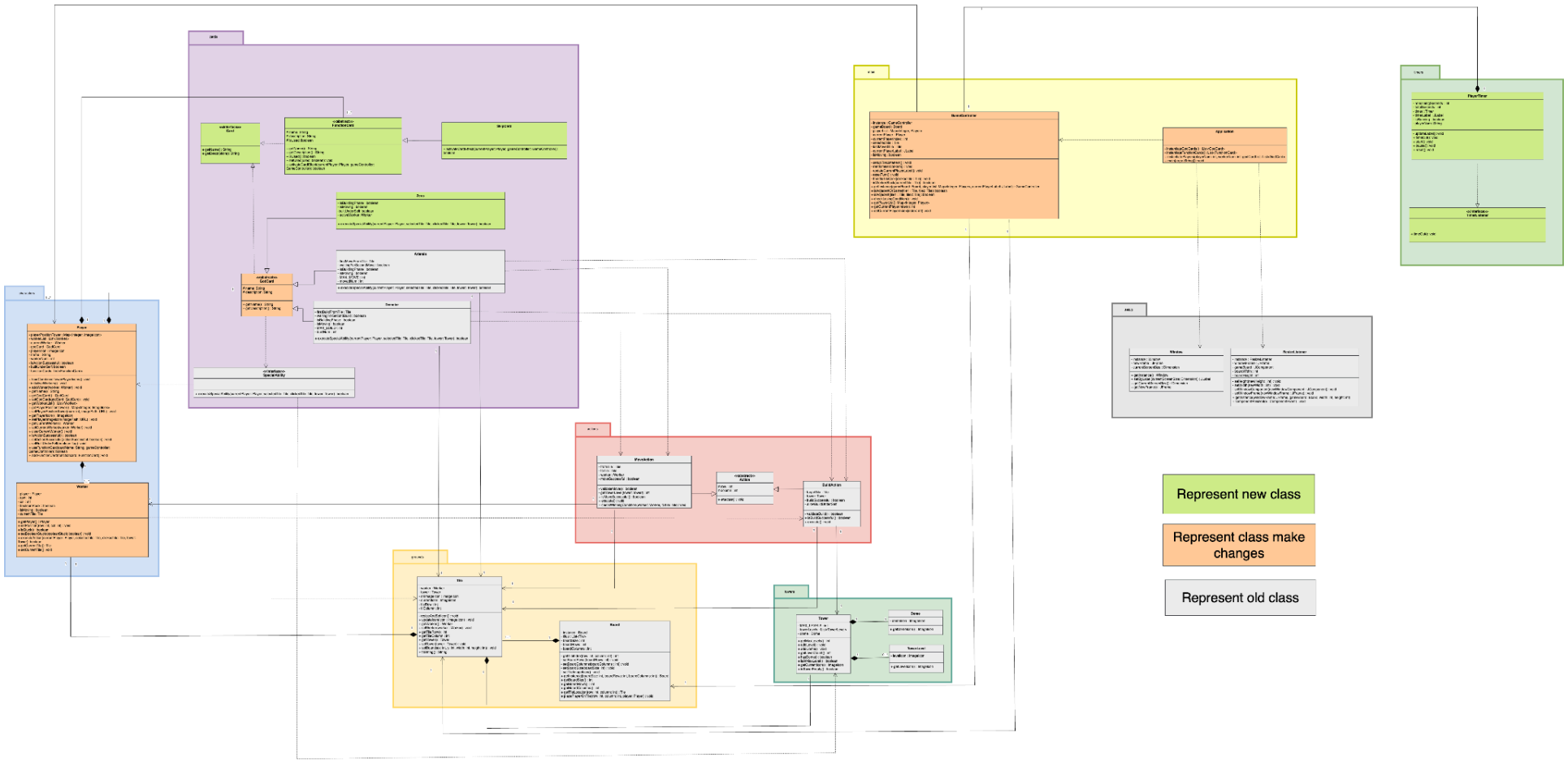


Diagram 3.6.2: UML Class Diagram Sprint 3

In Sprint 3, new classes such as PlayerTimer, TimerListener, Card, FunctionCard, SkipCard and Zeus (represented in green) were added to support new extensions and functions. Additionally, several existing classes such as Player, Worker, GodCard, GameController and Application were modified (represented in orange). Classes shown in gray remain unchanged from Sprint 2.

4.0 CRC (Class-Responsibility-Collaborator) Cards

This section shows 3 CRC (Class-Responsibility-Collaborator) cards based on the extensions that i had chosen in this sprint. The first row for each table represents the class of the CRC. The left column records the Responsibility of the class and the right column records the collaborators who work with or interact with the class.

4.1 PlayerTimer

PlayerTimer implements **TimerListener**	
Display countdown time for player turns	Jlabel
Track and update the remaining time	Timer, JLabel
Notify when time is up	TimerListener, JOptionPane
Reser timer	GameController
Start timer	GameController
Pause timer	GameController

Table 4.1 shows CRC cards of PlayerTimer.

4.2 SkipCard

SkipCard Extends **FunctionCard**	
Allow the player to skip the next player’s turn	Player, GameController
Ensure the card is used only once	GameController

Notify the player about the skip effect	JOptionPane
Prompt player to confirm use	GameController
Mark card as used	FucntionCard

Table 4.2 shows CRC cards of SkipCard.

4.3 Zeus

Zeus Subclass of **GodCard**	
Control worker movement and the building phase	MoveAction, Player, Worker
Use Zeus's special ability (built under self)	Tile, Tower, BuildAction, Player, GameController
Update game state and UI display	GameController, Tile, JOptionPane, ImageIcon
Trigger God's ability during the player's turn	Player
Display ability activation effect	JOptionPane
Extend god card behaviour	GodCard

Table 4.3 shows CRC cards of Zeus.

5.0 Design Rationale

5.1 Design Goal

This design breaks the system into smaller, focused classes to improve modularity and encapsulation. This makes the system easier to manage and maintain. Each class has a clear responsibility, which helps reduce complexity.

By following the DRY principle, the design minimises code duplication and encourages the reuse of common functionality. Besides, the system is also designed based on the SOLID principles, which help create well-structured, maintainable, and scalable code. These principles include:

- **S**ingle Responsibility Principle, which is to ensure each class has only one clear responsibility or purpose.
- **O**pen-Closed Principle, meaning classes are open for extension but closed for modification.
- **L**iskov Substitution Principle, which allows the derived subclasses to remain consistent with the expectations set by their parent classes.
- **I**nterface Segregation Principle, which promotes splitting interfaces so that each interface contains only the methods that are directly related to a specific responsibility.
- **D**ependency Inversion Principle, which promotes depending on abstractions rather than concrete implementations.

The aim is to keep the system flexible and adaptable to future changes.

Brief explanations and justifications of the UML Class Diagram iterations and the reasoning behind each change are provided below. A final UML diagram is also included to give an overview of the classes used in Sprint Three and how they are connected. For the final UML, all key aspects such as class responsibilities, relationships, alternative solutions considered, the final design decisions, reasons for the decision, along with their advantages and disadvantages, are thoroughly explained.

5.2 Class Description

The table below shows the design rationale for the classes.

Class Modified / Created	Roles and Responsibilities	Rationale
[package: cards]		
Card (interface)	Represent an interface for the all card types in the game. This interface provides	**Alternate Solution:**

<table>
<tr><td></td><td>a blueprint for the cards to define their name and description, allowing the game to treat all cards uniformly regardless of their specific behaviour.

Relationships:
1. It is implemented by different concrete card classes (e.g., FunctionCard, GodCard)</td><td rowspan="3">1. Create an abstract FunctionCard class with an abstract method called activeCardEffect().
2. Store all the common attributes that are needed for the child classes that are inheriting from it, such as ‘name’ and ‘description’.
3. Override the abstract method declared in the abstract FunctionCard class by implementing the actual logic of the card effect in each of the child classes, SkipCard, in this case.
4. Zeus class extends GodCard class (from Sprint 2) and inherits the common attributes.

Finalised Solution:
1. Create a Card interface to define a blueprint for all the cards, including god cards and function cards.
2. Add a getName() and getDescription() method in the interface to ensure that any class implementing it must provide the name and description.
3. Set FunctionCard as an abstract class.
4. Implement GodCard and FunctionCard classes to adhere to the interface (Card), so that the getName() and getDescription() are defined and customised by every card.
5. Create a new god card subclass, which is Zeus by extending from the GodCard abstract class.
6. Zeus is a concrete subclass of GodCard, which implements its specific special abilities in the executeSpecialAbility() method:
a. Zeus: Allows building underneath, but building a 3rd-level block underneath itself does not qualify as a win condition.

Reasons for Decision:
(Don’t Repeat Yourself Principle, DRY)
1. By using an abstract class like FunctionCard, all common features, such as storing and retrieving the name and description of function cards are implemented once and reused through all function subclasses. This ensures that any changes to these shared features can</td></tr>
<tr><td>GodCard
(abstract class)</td><td>Represents the general god card class. This abstract GodCard class is to store and hold the information of different god card instances. Each of the god cards has its own special ability to be played by the player. It provides a blueprint for its child classes, such as Artemis and Demeter.

Relationships:
1. Implements the SpecialAbility interface.
2. Implements the Card interface.
3. It extends by Zeus class.</td></tr>
<tr><td>Zeus
(extends abstract GodCard)</td><td>Zeus is an instance of a god card that can be played by a worker. Zeus has its special ability where it can build a block under itself in a single turn, with the condition that a worker building a 3rd-level block underneath itself does not qualify as a win condition.

Relationships:</td></tr>
</table>

<table>
<tr>
<td></td>
<td>1. It extends the abstract GodCard class. (This is to inherit all the needed functions and attributes for a god card.)
2. It associates with the Worker class.
3. It is dependent on the MoveAction.
4. It is dependent on the BuildAction.</td>
<td rowspan="3">be made in a single place, reducing duplication and the risk of inconsistencies throughout the codebase.

(Single Responsibility Principle, SRP)
1. Each class in the design has a clearly defined role and is responsible for only one thing. The FunctionCard handles the identity and data of the function cards, such as their name and description. It does not understand the behavior or gameplay mechanics. The Card interface defines the basic contract for all cards to expose their name and description. It keeps the interface focused and simple.
2. Zeus is only responsible for defining a single god's behaviour for their respective ability. The separation ensures that changes to how a god behaves in the game will not affect how data is stored and retrieved.
3. SkipCard is only responsible for defining its specific functional behavior. The separation ensures that changes to the card’s behavior will not affect how data is stored and retrieved.

(Open-Closed Principle, OCP)
1. The design is open for extension but closed for modification. To demonstrate, new function cards can be introduced simply by creating a new classes that extend FunctionCard. No changes are needed to existing classes, ensuring stability and reducing the risk of bugs. This is important in a game context, where new types of function cards like Swapcard or ReverseCard may be added in the future.

(Liskov Substitution Principle, LSP)
1. Any subclass of FunctionCard can safely be used interchangeably with the parent type without breaking the program. For example, the concept of polymorphism can be applied. FunctionCard can be used as the reference type when creating an instance of a subclass, such as (FunctionCard SkipCard = new SkipCard()). Other than that, the method declared in the abstract FunctionCard, such as getName() and</td>
</tr>
<tr>
<td>FunctionCard (abstract class)</td>
<td>Represents the general function card class. This abstract FunctionCard class is to store and hold the information of different function card instances. Each of the function cards has its own card effect to be played by the player. It provides a blueprint for its child classes, such as SkipCard..

Relationships:
1. Implements the Card interface.
2. It extends by SkipCard class.</td>
</tr>
<tr>
<td>SkipCard (extends abstract FunctionCard)</td>
<td>SkipCard is an instance of a function card that can be played by a player. SkipCard has an effect where it can skip the next</td>
</tr>
</table>

	player's turn, with the condition that each player can only use it once. **Relationships:** 1. It extends the abstract FunctionCard class. (This is to inherit all the needed functions and attributes for a function card.) 2. It depends on the Player class. 3. It depends on the GameController class.	getDescription() can be called without needing to know the specific function of cards. (Interface segregation Principle, ISP) 1. The Card interface defines only essential methods to avoid forcing unused behaviour on implementing classes. It provides a common abstraction for all card types and supports polymorphism while following the ISP. (Dependency Inversion Principle, DIP) 1. The Card serves as an abstraction layer. This allows the game logic, which is the high-level module, to depend on the interface rather than on the concrete implementation, like GodCard or FunctionCard. As a result, the direct dependency between the classes can be eliminated. Relying on the Card interface makes the system more flexible and easier to maintain. For example, a new card type like ItemCard can be added to the game without affecting or breaking the game, as long as the card implements the Card interface. **Advantages** 1. Each class has its own responsibility, which makes the code easier to update and extend. 2. Interface and abstract classes provide flexibility and modularity, which is important when additional function cards(e.g., ReverseCard) or other types of cards (e.g., Itemcard) are added in the future. 3. The inheritance encourages code to be reused, while interfaces and polymorphism allow for behavioural flexibility. **Disadvantages:** 1. The complexity of the code will be increased with the use of both abstract and interface classes. This is because the behaviour of the code is split across multiple layers of abstraction.
[package: timers]		

<table>
<tr>
<td>TimerListener (interface)</td>
<td>Represent an interface for the card of every cards. This interface provides a blueprint for the card to execute their special ability.

Relationships:
1. Implemented by PlayerTimer class.</td>
<td rowspan="2">Alternate Solution:
Create a single PlayerTimer class and combined all functionality of timer into the PlayerTimer class directly. Through this design, the PlayerTimer will be responsible for handling the all timer function including countdown logic. Do not split the timeout behaviour using any interface. Let PlayerTimer handle both timer control and timeout consequences in one class.

Finalised Solution:
1. Create a TimerListener interfave to define a generic timeout behavoiur contract using a timeOut() method. PlayerTimer class implements the TimerListener interface class to define specific actions when the time is out with showing a dialong and exit the game.
2. Use Swing’s Timer to control countdown logic in PlayerTimer, which uodates the JLabel and triggers the timeout callback when time runs out.

Reasons for Decision:

(Don’t Repeat Yourself Principle, DRY)
1. By separating timeout behaviour in an interface TimerListener, different timer behaviour can be defined without repeating countdown logic.
2. All common timer mechanisms, such as start, pause, reset, and label updates are written once in PlaterTimer and can be reused across different timeout use cases.

(Single Responsibility Principle, SRP)
1. TimerListener has only one responsibiliy, where it only define what happens when time runs out.
2. Other than that, PlayerTimer is responsible with controlling and displaying the countdown timer for player rounds.
3. This separation makes each class easier to understand and modify independently.</td>
</tr>
<tr>
<td>PlayerTimer (concrete class)</td>
<td>Represents a single cell on the board, which can hold either a Worker or a Tower object. It updates its image dynamically based on the changes to the worker or tower state.

Relationships:
1. Implements the TimerListener interface.
2. It is associated with Timer class.</td>
</tr>
</table>

		(Interface segregation Principle, ISP) 1. The TimerListener interface is small and focused, it only declare one method which is timeOut(). This prevents forcing unnecessary methods on implementing classes and keeps the interface highly reusable. **Advantages:** 1. The use of TimerListener supports flexible behaviour injection without changing existing code. 2. Makes the system easy to create different types of timers or reuse the countdown logic for different game scenarios. **Disadvantages:** 1. For simple games with only one type of timeout behavior, the interface might seem unnecessary.

5.5 Description of Cardinalities of the Relationships

The following are explanations for determining the cardinalities between two classes which are solely based on the Sprint 3 extensions.

Classes involved	Cardinality	Explanation of cardinality
Player -> FucntionCard	1 Player can have at least 1-to-many function cards. (1 - 1…*)	• 1 player can have at least 1-to-many function cards. The player is required to have at least one function card to interact meaningfully with the game's mechanics, such as skipping turns or reversing order. Therefore, the minimum cardinality of the FunctionCard would be 1 • The player can have many function cards as well as the cards are instantiated and stored in an ArrayList in the Player class. So, each player can hold multiple card instances. Hence, the maximum cardinality on the FunctionCatd side would be *. • Similarly, 1-to-many cards are assigned to 1 player. • The cardinality cannot be 1 to 2 because the Player class uses a collection to store cards, allowing flexible and dynamic card management throughout the game.

5.6 Decision of Inheritance

These are the justifications of using inheritance in our design.

Classes involved	Explanation about usage of inheritance in the design
• GodCard • Zeus	• Inheritance was used between the abstract class, GodCard and its subclass, Zeus. • All god cards share common attributes such as name and description as well as methods like executeSpecialAbility() that change the actions of the worker in the game. • These common attributes and methods can be defined once in the abstract class and can be directly inherited by the rest of the subclasses, thus avoiding code redundancy in several classes. • This also enables polymorphism, where different subclasses like Zeus can be accessed through GodCard references. Implementation of the logic is simplified, especially in the GameController class where it just calls the executeSpecialAbility( ) method without knowing the specific god card being used. • In addition, using inheritance in the design provides an extensible system where new god cards can be introduced to the game by extending from the abstract class and overriding the methods to perform their own logic without modifying the core functionalities in the other classes.
• FunctionCard • SkipCard	• The inheritance concept is applied in the design which involves the FunctionCard abstract class and its subclasses, which include SkipCard. • This subclass represents a specific function card that performs the skip action when played in the game. • The functionCard abstract class itself acts as a common structure that defines the attributes and methods which will be directly inherited buy the subclass. • Consistent behaviour can be maintained across all subclasses that represent different function cards. For example, the abstract method which is the applyCardEffect() in the FunctionCard abstract class can be overridden and the necessary logic can be implemented by the subclass and encapsulated without affecting the other classes. • This design reduces duplication of code and encourages flexibility, allows new function cards to be easily added to the game in the future.

• Card • FunctionCard • GodCard	• Both FunctionCard and GodCard implement the Card interface. • The Card interface introduces the method, such as getName() and getDescription(), that all types of cards must be implemented. • By implementing this interface, both FunctionCard and GodCard ensure a common structure that allows polymorphism and consistent interaction with various types of card by putting this interface. • This abstraction allows the game system to treat all cards uniformly, regardless of their specific behavior or type, makes the code management and interaction simplified. • Besides, implementing the Card interface promotes flexibility and extensibility, enabling new card types to be defined by simply implementing interface without altering the existing code.

5.7 Description of Design Patterns

These are the justifications of the design patterns used in our design.

Design Patterns Used	Classes Affected	Explanation
Strategy Pattern	• SkipCard • Func'tionCard • Card	How it is implemented: • The SpecialAbility interface is created and outlines a contract for any strategies like the special abilities or powers of each God card. It contains the interface method called executeSpecialAbility(). • The abstract class, GodCard implements the SpecialAbility interface and the executeSpecialAbility() method has to be overridden by each subclass of GodCard to implement their specific behaviour respectively. • For instance, Artemis will implement its logic of allowing the worker to move twice in one turn and Demter will implement its logic of allowing the worker to build twice in one turn. Reason why it is implemented: • Each god card possesses a special ability that can influence the game rules and mechanics. Having multiple conditional statements or excessive switch cases to manage each god card's specific logic can reduce code scalability and maintainability especially in the GameController class. • So, each god card's behaviour should be encapsulated in their own individual classes. This helps to promote code modularity and maintain a clear separation of responsibilities. • Through Strategy Pattern, the behaviour of that particular god card will be executed correctly depending on the god card selected for the player. The existing implementations in the GameController class do not have to be refined or modified as the god card chosen will be instantiated before invoking the executeSpecialAbility() method which is polymorphic, hence allowing interchangeability.

		• Furthermore, this design pattern allows the extension of new God cards with distinct abilities in the future. They can be added into the existing codebase directly just by implementing the SpecialAbility interface and overriding the aforementioned interface method without disrupting the overall game logic and impacting the other classes.

5.6 Human Values - Influential

The human value – Influential describes a personal success through having an impact on others. It is important in competitive conditions because it reflects strategic thinking and control. Players who value influence seek control results and demonstrate their dominance by making smart decisions.

In this game, the SkipCard shows the Influential value. A skip card allows a player to skip the next player's turn. This directly impacts on the next player's progress and allows the current player to control the flow of the game. It shows the influence and success through timing and strategy.

To achieve this, the SkipCard class has been created with extended as a subclass of FunctionCard, and the game logic in the GameController to handles the skip mechanics. If the player uses the card in the game, the next player is forced to miss a turn. The player and Sprint 3 marker can experience the value when the skip card is played. This visible impact clearly shows the use of influence, fulfilling the purpose of the card and reinforcing the Influential human value.

6.0 Description of Design Patterns

These are the justifications of the design patterns used in our design.

Design Patterns Used	Classes Affected	Explanation
Template Pattern	• FunctionCard • SkipCard	**How it is implemented:** • The FunctionCard abstract class acts as a template and defines the common structure for all function cards. It includes shared attributes like name and description and declares an abstract method activateCardEffect(), which serves as a template design pattern to be implemented by each specific function card subclass. • FunctionCard is extended by subclasses like Skipcard, which also offer their version of the activateCardEffect() method. For example, when the card is utilised, Skipcard overrides the way to skip the turn of the next player. • This design allows all function cards to be handled as FunctionCard type, but each card can still exhibit its unique behavior when executed through polymorphism. **Reason why it is implemented:** • Each function card performs a different behaviour in the game. If the behavior logic of all cards is focused in one class and controlled by a large number of conditional judgments (such as if or switch), it will not only make the code difficult to maintain but also reduce scalability. • By using the template design pattern, the common behavior logic is focused in the FunctionCard abstract class, while the specific logic of each function card is implemented by its own subclass (such as SkipCard). This approach helps separate responsibilities, reduce code duplication and improve code maintainability. • Other than that, this design also makes it easy to add a new function card in the future. Developers only need to inherit the Functioncard and implement

		the activateCardEffect() method to add new cards without modifying the existing game logic or other card classes. This adheres the open-closed principle (OCP) and improves the overall flexibility and scalability of the system

7.0 Executable Description

Below shows the instructions to create a JAR file in IntelliJ IDEA (macOS).

You can create it by watching this video or following the steps:
https://drive.google.com/file/d/1XM8m3xfKOC33M6FS5Jmb7HpK-T1OmBxs/view?usp=drive_link

Steps:

1. Open the project in IntelliJ IDEA.

2. Go to the resources folder
   2.1. Right click → navigate to Mark Directory as → "Resources Root"

3. Click File → Project Structure
   3.1. Go to Artifacts and click the "+" sign
   3.2. In the "Add" popup, click JAR and then select "From modules with dependencies…".
   3.3. For the Main class, click on the folder icon and then select "Application of sprint3implementation…".
   3.4. Click "OK".

4. Click Build → Build Artifacts..
   4.1. Click "Build"

5. You will see the JAR file in "out" folder

8.0 Reflection

For the extension for Zeus (new god card), the difficulty is easy. This is because in Sprint 2, there is already defined a Godcard abstract class and Zeus class could easily extend from it. The polymorphic design and use of inheritance allowed it to override the executeSpecialAbility() method with custom logic without affecting other card types.

9.0 Important Links

9.1 Draw.io

https://app.diagrams.net/#G1beNQkuB7S2RBM2jzwYziAilOEyRbWae-#%7B%22pageId%22%3A%22m-ngLz56KHzYYwQyDyMN%22%7D

9.2 Contribution Log

https://docs.google.com/spreadsheets/d/1A0S95H4KA7uKyUwhTPJlzqZwaLYjGHFG/edit?gid=316231838#gid=316231838

9.3 Git repository

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2025/assignment-individual/xwon0021

9.4 Link to Sprint Three document

https://docs.google.com/document/d/1JIPFy1swR03W7Qk3K_8cF4vryeUzay_D/edit

9.5 How To create JAR file

https://drive.google.com/file/d/1XM8m3xfKOC33M6FS5Jmb7HpK-T1OmBxs/view?usp=drive_link